

Embedded Operating Systems and Architectures

Stridi Simone

Anno Accademico 2026/2027

Contents

1	Introduction and Toolchain	3
1.1	Technical Foundations & Toolchain	3
1.1.1	From Source Code to a Running Program	3
1.1.2	Why Cross-Compilation?	3
1.1.3	Host, Target, and Emulator	4
1.1.4	Bare Metal Means We Provide the Runtime	4
1.1.5	Preprocessing: Assemble the Translation Unit	5
1.1.6	Compilation: C Becomes Target Instructions	6
1.1.7	Assembly: Instructions Become Relocatable Bytes	6
1.1.8	Linking: Creating the Executable	7
1.1.9	What's Inside an ELF File	8
1.1.10	Sections Explain the Static Layout	9
1.1.11	File Size is Not RAM Usage	10
1.1.12	Debug Information for Host Tools	11
1.1.13	What Make Actually Does	11
1.1.14	QEMU Runs a Board Model	14
2	Boot and CPU Configuration	17
2.1	Introduction to Low-Level CPU Control	17
2.2	Memory Technologies in Embedded Systems	17
2.2.1	SRAM (Static RAM): The Working Memory	17
2.2.2	NOR Flash: Execute-in-Place Memory	18
2.2.3	NAND Flash and Block Devices	18
2.2.4	Applying Constraints to C Code	18
2.2.5	The Cortex-M3 Memory Map	19
2.2.6	The Linker Script: Mapping the Chip	19
2.2.7	Run Address vs. Load Address: The <code>.data</code> Section	22
2.2.8	BSS, Heap, and Stack Layout	22
2.2.9	The Role of Linker-Defined Symbols	22
2.2.10	Reading the Compilation Results	23
2.2.11	Which Section Does a Declaration Go To?	24
2.2.12	Advanced Memory Layout and Custom Sections	25
2.2.13	Managing Memory Size Constraints	26
2.2.14	Reset: The Hardware Contract	26
2.2.15	The Vector Table: Controlling the Boot Process	27
2.3	Taking Control of the CPU	28
2.3.1	Startup Code: Assembly vs. C	28
2.3.2	Remote Debugging in a Bare-Metal Environment	28

2.3.3	Cortex-M3 Programmer's Model: The Registers	30
2.3.4	ARM Assembly Basics: A Load/Store Architecture	31
2.3.5	Control Flow: Decisions and Loops	32
2.3.6	Assembler Directives	33
2.4	Writing the Startup Script (Reset_Handler)	33
2.4.1	Steps 1 and 2: Mask Interrupts and Set SP	34
2.4.2	Step 3: Copy .data from Flash to RAM	34
2.4.3	Step 4: Zero the .bss Section	36
2.4.4	Exceptions and Faults: Completing the Vector Table	37

Chapter 1

Introduction and Toolchain

1.1 Technical Foundations & Toolchain

1.1.1 From Source Code to a Running Program

The goal of this phase is to understand how to compile and run an application step by step, working directly with commands without relying on an IDE (like Visual Studio Code). Often, development environments hide the real compilation process behind a simple "Play" button, but when developing operating systems, it is essential to understand what actually happens under the hood.

The toolchain is a pipeline composed of several small tools, where the output of one becomes the input of the next:

- **C Preprocessor:** manages and expands directives like `#include` and `#define`.
- **Compiler:** takes the preprocessed code and generates assembly code (`.s`).
- **Assembler:** converts the assembly into machine code, producing object files (`.o`).
- **Linker:** concatenates matching sections, resolves symbols to final addresses, and through an `.ld` script, generates the final binary image (the ELF image).

1.1.2 Why Cross-Compilation?

Developing for embedded systems introduces an architectural problem: the computer we are writing the code on (Host) is different from the machine where the code will be executed (Target)[cite: 1]. The host is typically an x86-64 or Apple Silicon machine, whereas our target is an ARM Cortex-M3 processor[cite: 1].

A normal `gcc` compiler installed on the computer emits code optimized for the machine it runs on, which makes it useless for our purpose[cite: 1]. For example, a recent Mac's native compiler would generate code for a 64-bit ARM architecture, while our target board requires code for a 32-bit ARM architecture.

For this reason, a **cross-compilation toolchain** (specifically `arm-none-eabi-*`) is required[cite: 1]. It runs on the host but generates code for the target's ISA (Instruction Set Architecture)[cite: 1].

The `none-eabi` nomenclature is crucial to understand the environment we are working in:

- **none:** indicates the absence of a host operating system on the target (bare-metal development)[cite: 1].
- **eabi:** indicates the use of the standard embedded ARM ABI (Application Binary Interface)[cite: 1].

Working in a "bare-metal" environment means not having the standard libraries typical of desktop operating systems: there will be no `printf` to a console, no `malloc` dynamic allocation supported by an OS, and no `_start` function from a C library unless we explicitly provide it[cite: 1].

Checking the Toolchain: Host vs Target

To concretely verify the difference between compilers, we can simply query the toolchain from the terminal:

- By running `gcc -dumpmachine`, you get the architecture of your host machine (e.g., a 64-bit architecture for a recent desktop system)[cite: 1].
- By running `arm-none-eabi-gcc -dumpmachine`, the output will clearly indicate a 32-bit ARM architecture followed by the `none` environment.

Getting two completely different target strings for the same command typed on the same computer demonstrates the fundamental concept of a cross toolchain[cite: 1]. When writing an operating system from scratch, it is imperative to use this type of compiler, generate the binary file, transfer it to the target machine, and finally run it.

1.1.3 Host, Target, and Emulator

During the development of software for embedded systems, it is essential to keep three key roles distinct in the compilation and testing process[cite: 1]:

- **Host:** This is the PC we work on (e.g., our laptop) and where the text editor, toolchain, and other development tools are executed[cite: 1].
- **Target:** This is the physical hardware destination machine (in our case, a board with an ARM Cortex-M3 processor) for which the executable (the ELF image) is generated[cite: 1].
- **Emulator:** This is a virtual machine that simulates the target's hardware via software directly on our host[cite: 1]. In our case, lacking the physical board, we will use **QEMU** to test the execution of the programs[cite: 1].

1.1.4 Bare Metal Means We Provide the Runtime

When running a program on a desktop computer, the operating system and the C runtime take care of preparing the memory and starting the program automatically[cite: 1]. In a bare-metal environment, at boot time, no such service exists[cite: 1].

To execute even just the first instruction of our `main()` function, it is necessary to have startup code that takes care of initializing the entire environment. Specifically,

the startup code must provide the vector table, initialize the `.data` and `.bss` memory sections, call constructors, and define what happens when `main()` ends[cite: 1]. Even though for today's tests we will use software that already performs many of these initializations for us, it is crucial to know that without these steps, the hardware would not know how to execute the program.

1.1.5 Preprocessing: Assemble the Translation Unit

When we start compiling (e.g., by typing the classic `gcc` command), the process is not monolithic but is divided into very distinct phases. The very first phase is handled by the **C preprocessor**. The preprocessor analyzes the C source code looking for directives (commands starting with the `#` character, such as `#include` or `#define`) and expands these macros before the compiler actually sees the program[cite: 1].

This is purely text substitution, without any machine code generation[cite: 1]. For example, upon encountering an `#include` command, the preprocessor locates the specified file and essentially "copies and pastes" it into the source file, replacing the original directive. The result of this operation is a *translation unit*: a single file of pure C code with all macros and headers already expanded[cite: 1].

Try It: Preprocess Only

We can instruct the toolchain to run *only* the preprocessing phase by using the `-E` flag. The command to run in the terminal is the following[cite: 1]:

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding  
-E src/main.c -o build/main.i
```

In this string, we provide several options to describe to the toolchain the characteristics of the machine we actually have available[cite: 1]:

- `-mthumb` and `-mcpu=cortex-m3`: these indicate, respectively, to use the Thumb instruction set and that the target CPU is a Cortex-M3[cite: 1].
- `-nostartfiles` and `-ffreestanding`: these signal that we are working in a bare-metal environment, meaning without standard operating system libraries or default startup files[cite: 1]. These options will be the default for our projects.
- `-E`: commands the compiler to stop immediately after preprocessing[cite: 1].

If we open the generated output file (`main.i`), we will notice fundamental changes compared to the original source:

1. All text comments present in the source have been removed by the preprocessor.
2. The `#include <stdint.h>` directive has disappeared, entirely replaced by the expanded text (the typedef definitions) contained in that header file[cite: 1].
3. The original C code of the `main()` function has remained textually identical at the bottom of the file[cite: 1].

This demonstrates that the actual compiler, in subsequent steps, will never read our original `main.c`, but will work on this "translated" file (`main.i`), which can become huge if there are many files to include.

1.1.6 Compilation: C Becomes Target Instructions

Once preprocessing is finished, the compiler itself comes into play. The role of this step is to check the C syntax and translate the preprocessed code into **assembly** code specific to our target architecture[cite: 1]. All the options we provided earlier (such as `-mcpu`) are needed precisely by the compiler to know exactly which instructions to generate for 32-bit ARM microprocessors.

Compilation and Target Architecture

There are many 32-bit ARM microprocessors available, so it is necessary to specify the exact model being used, which in our case is the Cortex-M3. This specific CPU supports the Thumb instruction set (Thumb-2)[cite: 1]. Therefore, the compiler must be instructed precisely about the target CPU by providing specific options[cite: 1]. Finding the correct configuration involves matching the hardware datasheet specifications with the extensive GCC compiler documentation. Once the correct configuration is established and understood, it can be seamlessly applied to all future projects running on that same target machine.

Try It: Stop After Compilation

To observe the compilation phase without proceeding further in the pipeline, we can run the compiler using the `-S` flag (instead of the `-E` flag used for preprocessing)[cite: 1]. The `-S` option instructs the toolchain to run the preprocessor, compile the code, and stop immediately after producing the assembly version of the C program[cite: 1].

The output is typically stored in a file with a `.s` extension, such as `main.s`. Opening this file reveals that the original high-level C program has been translated into low-level ARM assembly instructions[cite: 1]. These instructions are written using human-readable mnemonics, not raw binary code, proving that the compiler's role at this specific stage is purely translation into assembly language[cite: 1]. While reading these files is not strictly required for this course, it is crucial to know that this intermediate translation step always takes place.

1.1.7 Assembly: Instructions Become Relocatable Bytes

Once the assembly version of the code is generated, the next tool in the chain is the **Assembler**. Its role is similar to a compiler, but it specifically takes the assembly code (`.s`) and translates it into binary machine code[cite: 1].

The output of this phase is an **object file** (identifiable by the `.o` extension). However, it is important to emphasize that the binary code contained in an object file is still not executable firmware[cite: 1]. Because projects are usually composed of multiple source files, the assembler processes each file in complete isolation. As a result, the generated object files contain machine-code bytes, section names, and symbol definitions, but they still lack proper, final memory addresses[cite: 1]. They may also contain unresolved references to code located in other object files (for instance, `startup.o` referring to `main`), which cannot be resolved at this stage[cite: 1].

Try It: Assemble to an Object File

To run the complete compiling chain up to the end of the assembly phase (executing preprocessing, compiling, and assembling sequentially), the `-c` flag must be used[cite: 1]. We can apply this to the different source files in our project:

- `arm-none-eabi-gcc [...] -c boot/startup.c -o build/boot/startup.o`[cite: 1]
- `arm-none-eabi-gcc [...] -c src/main.c -o build/src/main.o`[cite: 1]

By doing this, we generate the object files `startup.o` and `main.o`. Unlike the previous `.i` and `.s` files, these object files represent the first output in the pipeline that is entirely in binary format[cite: 1]. Opening them with a standard text editor will reveal unreadable characters, as they are no longer in a human-readable format.

Inspecting Object Files

To understand what is inside an object file, we can use a tool provided by our toolchain called `arm-none-eabi-nm`[cite: 1]. This tool reads the `.o` files and extracts information in a human-readable way, specifically focusing on "symbols"[cite: 1].

Symbols represent the memory addresses that our program uses, which at this assembly stage are still left open and undefined[cite: 1]. For example, if we run the tool on our object files, we can look for the `main` function:

- In `startup.o`, there is a call to the `main` function, but it is not defined there. The tool will label this symbol with a **U** (Undefined), meaning it is referenced but its address is unknown[cite: 1].
- In `main.o`, the function is actually written and defined. The tool will label this symbol with a **T** (indicating it is defined in the `.text` section)[cite: 1].

This separation is completely normal: the files are compiled in isolation, and it will be the linker's job to connect the "U" to the "T" and assign the final memory addresses[cite: 1].

1.1.8 Linking: Creating the Executable

As established, object files are not runnable firmware[cite: 1]. In ARM architecture, every assembly line corresponds to a 32-bit instruction with a fixed binary encoding. However, whenever an instruction needs to reference a memory address (like calling a function), those bits are left blank and filled with symbols instead.

To convert these isolated objects into a final executable, we need a last step: the **Linker**[cite: 1]. The linker takes all the independent object files, concatenates matching sections, resolves every symbol to a final address, and patches all the references[cite: 1]. The output is an executable file formatted as **ELF** (Executable and Linking Format), which is the standard format originally designed in Unix and used globally (unlike the proprietary `.exe` format used by Windows)[cite: 1].

The Linker Script

When compiling a program on a standard PC, the operating system manages memory automatically, so the linker already knows what to do[cite: 1]. However, in a bare-metal embedded system, there is no operating system to rely on[cite: 1].

Therefore, we must explicitly instruct the linker on how to organize the different pieces of code in memory using a **Linker Script** (usually a `.ld` file)[cite: 1]. This script specifies the physical addresses of the hardware, detailing exactly what goes into the Flash memory (e.g., `.text`, `.rodata`) and what goes into the SRAM (e.g., `.data`, `.bss`, `stack`)[cite: 1]. Without this script, the toolchain wouldn't know the chip's specific memory map[cite: 1].

Try It: Link the Objects

If we want to run the compiler without stopping at intermediate steps and link the project, the command becomes more complex because we must include the linker script and all the necessary object files[cite: 1].

The command looks like this: `arm-none-eabi-gcc [...] -T scripts/mps2_m3.ld [...] build/boot/startup.o build/src/main.o -o build/scratch-os.axf`[cite: 1]

Here, the `-T` flag is crucial because it tells the compiler which linker script to use (`scripts/mps2_m3.ld`)[cite: 1]. After running this, we obtain our final, linked ELF image (`scratch-os.axf`). If we inspect it again with `arm-none-eabi-nm`, the `main` symbol will no longer be marked as "U" (undefined), but will have a finalized memory address assigned by the linker[cite: 1].

The Linker Script and the Map File

To perform the final link, we must tell the compiler which linker script to use via the `-T` option[cite: 1]. The project already includes a specific linker script (e.g., `mps2_m3.ld`) tailored for the specific board we are emulating[cite: 1]. The linker script is fundamental because it knows the exact memory layout of the board: how much Flash memory and RAM are available, their specific addresses, and where each symbol should be placed (e.g., instructing the toolchain to place executable code in Flash and variables in RAM)[cite: 1].

Another important option passed to the toolchain during this phase is the instruction to generate a **map file** (typically using a `-Map` flag)[cite: 1]. This file contains detailed information about the linker's final output and where everything was placed in memory, which is extremely useful for debugging purposes[cite: 1].

The final output of this entire linking process is an executable binary file[cite: 1]. In the ARM ecosystem, it is common practice to use the `.axf` extension for these files, although technically any extension can be used to denote the binary output[cite: 1]. This is the actual file that, in theory, can be flashed onto the physical board to run the program[cite: 1].

1.1.9 What's Inside an ELF File

The binary executable we generate is formatted as an **ELF** (Executable and Linking Format) file[cite: 1]. This standard format takes the binary code and organizes it into discrete blocks called **sections**[cite: 1].

If you inspect an ELF file internally, you will find an ELF header at offset 0[cite: 1]. This header acts as a directory, explicitly stating which sections are present in the file, at which precise byte each section starts, and how many bytes it contains[cite: 1]. It also points to the entry point of the program[cite: 1].

Execution Model: Desktop vs. Bare-Metal

The way an ELF file is loaded and executed depends heavily on the target environment:

- **Desktop Computers (Linux/macOS):** Every program is essentially an ELF file (note that Windows uses a completely different proprietary format, not ELF)[cite: 1]. When you run a program on these systems, a component of the operating system called the *loader* opens the ELF file, reads every section, and loads them into a free area of RAM before starting the execution[cite: 1].
- **Embedded Bare-Metal Boards:** In this environment, there is no operating system or loader[cite: 1]. Instead, the ELF file is transferred to the board via a special cable using a flashing program that reads the ELF file and writes the data directly into the proper locations in the board's Flash memory[cite: 1]. Once the board is powered on, the program is already in memory and runs immediately[cite: 1].
- **Emulation (QEMU):** Since we are not using a physical board in this course, our emulator performs this task[cite: 1]. It mimics the physical process by taking the ELF file, placing it into the simulated memory map, powering on the virtual machine, and starting the execution[cite: 1].

Try It: Read the ELF Header

Since the ELF file is pure binary, we cannot simply open it in a text editor to understand its contents[cite: 1]. Instead, we use a tool provided by the cross-compilation toolchain called `arm-none-eabi-readelf`[cite: 1].

By running the command `arm-none-eabi-readelf -h`, we instruct the tool to extract and print the header of the ELF file in a human-readable format[cite: 1]. The output includes crucial information such as the compiler used, the entry point address, and the sizes of the different sections[cite: 1].

This tool is invaluable for debugging[cite: 1]. If a compiled program is not working correctly, the issue might stem from a faulty linker script that placed sections at the wrong memory addresses[cite: 1]. By inspecting the ELF file with `readelf`, a developer can manually verify if the sections are actually located at the expected addresses.

1.1.10 Sections Explain the Static Layout

Operating system development is uniquely challenging; while the algorithms might be simpler than those used in artificial intelligence, the bare-metal environment is incredibly unforgiving, and a single mistake can crash the entire system. When debugging, tracing the problem often leads to analyzing the compiled output itself.

Every ELF file organizes the program into predefined logical blocks called sections. At a minimum, there are five crucial sections to understand[cite: 44]:

- **.isr_vector**: The interrupt vector table[cite: 44]. In bare-metal programming, developers must explicitly define the routines executed for each hardware interrupt, and this table sits at the start of the Flash memory[cite: 44].
- **.text**: Contains the executable machine instructions (the actual code)[cite: 44]. Since this never changes at runtime, it is placed in Flash[cite: 44].
- **.rodata**: Contains read-only constants, which are also placed in Flash[cite: 44].
- **.data**: Contains writable variables with initial values (global initialized data)[cite: 44]. This section occupies both Flash and SRAM: the initial values must be saved in the binary file (Flash), but at startup, they are copied into SRAM so they can be modified during execution[cite: 44, 46].
- **.bss**: Contains writable zero-initialized (or uninitialized) variables[cite: 44]. These do not require physical storage space in the ELF file on Flash, but they necessitate space reservation in the SRAM during execution[cite: 44, 46].

Try It: List the Sections

To inspect these sections, the toolchain provides the `arm-none-eabi-objdump` command[cite: 45]. While `readelf` provides high-level header information, `objdump` delves directly into the binary code itself[cite: 45].

By running `arm-none-eabi-objdump -h build/scratch-os.axf`, you can display all the different sections, indicating their sizes and the memory addresses (VMA for run address, LMA for load address) where they will be located[cite: 45]. This is an essential debugging tool for OS designers: if the board crashes or behaves unexpectedly, one can inspect the binary file to trace expected addresses and verify exactly where the code is attempting to jump[cite: 45].

1.1.11 File Size is Not RAM Usage

There is a connection between the size of the ELF file and the memory the program will use, but it is not a direct 1:1 relationship[cite: 46]. Because sections like `.bss` consume RAM but have no initial bytes stored in the ELF file, simply looking at the file size on disk is misleading[cite: 46].

Memory budgets are strictly divided:

- **Flash usage**: Consists of code (`.text`), constants (`.rodata`), and the `.data` initial image[cite: 46].
- **RAM usage**: Consists of `.data`, `.bss`, stack, and heap reservation[cite: 46].

Try It: Flash vs. RAM Totals

To precisely calculate memory occupation, we can run the `arm-none-eabi-size` command on our ELF file[cite: 47]. This command reads the file and outputs the sizes of the `text`, `data`, and `bss` columns, giving exact information on the number of bytes the program will actually use in memory[cite: 47]. If a program crashes unexpectedly, it may simply be due to attempting to use more memory than the board physically has, making this command a vital first step in debugging[cite: 47].

1.1.12 Debug Information for Host Tools

When an operating system is finalized and released, information about the original source code is typically stripped from the binary to reduce size and protect intellectual property. However, during development and debugging, it is highly beneficial to generate binary files that contain debugging information[cite: 48].

Specific compiler options can embed source-level debug metadata inside the ELF[cite: 48]. This does not mean the target machine understands C; it still only runs machine instructions[cite: 48]. Instead, this metadata allows host-side tools (like QEMU or a debugger) to associate a specific binary instruction address with a corresponding C file line, function, or variable[cite: 48]. This bridge between high-level C code and low-level execution makes investigating crashes possible without altering the basic boot image[cite: 48].

1.1.13 What Make Actually Does

While it is crucial to understand how to compile an embedded program manually, step by step, doing so in practice is extremely inefficient[cite: 49]. Remembering every compiler flag, every linker script option, and listing every single C file becomes impossible for complex projects (for instance, the FreeRTOS kernel consists of hundreds of C files)[cite: 49].

To automate the compilation process, we use tools like **Make**, which is the standard toolchain automation used by Linux and FreeRTOS[cite: 49]. A **Makefile** is essentially a text-based script that defines a set of rules to compile many files[cite: 49].

Targets Model Desired Outcomes

The **Makefile** works by defining **targets** and the actions required for each target[cite: 50]. A target is a named file or action that Make knows how to produce[cite: 50]. For example, if you have a target named **all**, you can simply type **make all** in the terminal, and the tool will execute all the instructions associated with that target[cite: 50]. Some targets, like **clean** or **run**, are usually actions rather than actual files, and they are declared as **.PHONY** to prevent conflicts with a same-named file[cite: 50].

Make is also powerful because it tracks **prerequisites** (the dependency graph)[cite: 50, 54]. If you specify that the linking step requires all object files to be created first, Make will search for the rule to generate those object files if they do not exist[cite: 50, 54]. It works backward, starting from the desired result and executing all the necessary preceding steps[cite: 50, 54].

Prerequisites Capture Rebuild Logic

Another key feature of Make is its ability to rebuild a target only if a prerequisite is newer than the target itself[cite: 49, 51]. For example, if you change one variable in a massive project (like the Linux kernel, which can take hours to compile), you do not want to recompile everything[cite: 49, 51]. Make identifies that only one **.c** file has changed, recompiles that specific file into an object file, and then simply re-runs the linker to update the final ELF image[cite: 49, 51].

To identify if something has changed, Make uses the last modified timestamp of the files[cite: 51]. However, this can sometimes lead to issues. If you are using version control (like Git) or syncing folders via cloud services (like OneDrive or Dropbox), file timestamps can become corrupted or mistakenly set in the future. In these cases, you might modify a file, but Make will refuse to recompile it because it believes the target is already up-to-date. If you encounter strange build behaviors, checking the timestamps of your files is an excellent troubleshooting step.

Variables and Compiler Flags

The **Makefile** uses a simple language that allows the definition of variables[cite: 49]. Variables are incredibly useful because they centralize configuration: you define them once and use them throughout the script[cite: 49].

For instance, the compiler flags encode crucial information such as the target architecture, warning policies, optimization levels, include paths, and bare-metal assumptions[cite: 52]. These are not just personal preferences, but a strict contract for the build[cite: 52]. Common variables include[cite: 52]:

- **CC**: Chooses the cross compiler (e.g., saving **arm-none-eabi-gcc** into this variable so that switching to a different compiler only requires changing one line)[cite: 52].
- **CFLAGS**: Governs the compilation of each source file[cite: 52].
- **LDFLAGS**: Governs memory layout and the final link[cite: 52].

Build Directories Keep Artifacts Separate

In a well-structured **Makefile**, it is common practice to save the generated executable file and intermediate objects in a dedicated directory, often named **build**[cite: 53].

Using a build directory provides several benefits[cite: 53]:

- Source directories remain readable and uncluttered[cite: 53].
- The **clean** target can remove generated files safely without risking the deletion of source code[cite: 53].
- Different configurations can coexist in separate build folders[cite: 53].
- A fresh build tests whether the dependencies are complete[cite: 53].

It is crucial to remember never to manually edit the generated object files or ELF bytes in the build directory in an attempt to change program behavior[cite: 53]. All changes must originate from the source code.

Variables and String Manipulation in Make

Using variables in a **Makefile** is not strictly mandatory; you could write raw commands by skipping variables entirely if your script consists of just one command[cite: 52]. However, it is a highly recommended practice for clean and maintainable code. By saving compiling information (such as the compiler name, the target executable

name, or the `build` directory path) into variables, you create a single point of entry[cite: 52, 53]. If, for instance, you decide to switch to another compiler, you only need to change one line, and the rest of the file will still function perfectly[cite: 52]. Variables in Make are essentially strings, and you can perform both variable assignment and variable concatenation[cite: 52].

Make also possesses a powerful language for processing strings and performing replacements, somewhat similar to regular expressions. This is particularly useful when managing source files. Instead of manually writing the compiling command for 100 different C files, you can list all the `.c` files in a `SOURCES` variable[cite: 54]. Then, you can use a short string substitution command to automatically generate the list of object files (`OBJECTS`)[cite: 54]. For example, the command can parse the `SOURCES` string, split it by slashes to remove the original directory (e.g., `boot` or `src`), prepend the `build` directory, and replace the `.c` extension with `.o`[cite: 54]. This automation is why Make is an indispensable tool in large projects[cite: 54].

Defining Targets and Rules

A `Makefile` is composed of targets (what you want to execute) and the instructions to execute them[cite: 50, 54]. To prevent Make from getting confused between actual files and actions, targets that do not produce a file (like `clean` or `run`) must be explicitly declared as targets using the `.PHONY` directive[cite: 50]. In our project, we define three main targets[cite: 50]:

- `all`: Compiles everything[cite: 50].
- `clean`: Deletes the build folder, allowing compilation to restart from scratch[cite: 50, 53].
- `run`: Runs the QEMU emulator and executes the program[cite: 50, 54].

Rules in Make define how a target is built. The syntax involves a colon (`:`), with the target (or the file to produce) on the left side, and the dependencies (the prerequisites) on the right side[cite: 54]. For example, the rule to generate the final executable (`$(BIN)`) depends on the object files (`$(OBJECTS)`) being available[cite: 54]:

```
$(BIN): $(OBJECTS)[cite: 54]
```

When you run `make`, the tool will check if the `$(BIN)` file is present[cite: 54]. If it is not, or if any file used to generate it has a modified timestamp, Make must generate it[cite: 51, 54]. To do so, it looks at the right side of the colon[cite: 54]. If the required object files are missing, Make will search backward for another rule capable of generating those `.o` files from the original `.c` files[cite: 54]. Once the object files are ready, Make will finally execute the compiler command (using all the variables previously defined for the compiler name, flags, and linker options) to produce the final executable[cite: 54].

A Disciplined First Build Loop

The `Makefile` operates on a dependency graph[cite: 54]. For instance, if the target executable (`$(BIN)`) cannot be generated because the object files (`.o`) are missing, Make searches for the rule that dictates how to create them[cite: 54].

This rule typically looks like this: `$(BUILD_DIR)/%.o: %.c`[cite: 54]

This means that for every `.o` file on the left side, the dependency is the corresponding `.c` file on the right side[cite: 54]. If the `.o` file is missing or outdated, this rule is triggered[cite: 54]. The actions associated with this rule usually involve:

1. Creating the `build` folder if it does not already exist.
2. Running the compiler with the `-c` flag, which instructs it to compile and stop before linking[cite: 54].

When you simply type `make`, the tool executes these rules in reverse order: it checks for the final ELF file, realizes it needs the object files, realizes it needs to compile the source files, and then executes the steps sequentially from the bottom up. This provides a highly disciplined build loop: you can run `make` to build the ELF, inspect the terminal for any compiler errors, run the generated ELF, and compare the observed output with what you expect[cite: 55].

1.1.14 QEMU Runs a Board Model

The final crucial target in our `Makefile` is the `run` command[cite: 54]. The rule for `run` depends on `$(BIN)`: the finalized ELF file must be available before execution can begin[cite: 54]. If the file is present, Make executes QEMU (`qemu-system-arm`)[cite: 56].

The emulator requires several parameters to function correctly:

- `-machine mps2-an385`: This tells QEMU which specific hardware board it needs to emulate, ensuring the CPU and memory map match our Cortex-M3 target[cite: 56].
- `-kernel`: This parameter passes the generated ELF file to the emulator[cite: 56].

Other options might include disabling the graphical screen (since we only need the terminal output) and enabling semi-hosting to emulate a debugging cable attached to the board.

When this command runs, QEMU emulates the entire physical process: it acts as if a program has flashed the ELF into memory, the power is turned on, and the board starts executing[cite: 56].

QEMU Output is an Early Hardware Check

Running the project in QEMU serves as a vital early hardware check[cite: 57]. If the terminal prints the expected output (e.g., "toolchain OK"), it proves that several complex layers are perfectly aligned[cite: 57]:

- The cross compiler successfully emitted valid target instructions[cite: 57].
- The linker placed the startup code and application at usable addresses[cite: 57].
- The reset reached the `Reset_Handler`, which correctly reached `main()`[cite: 57].

- The selected output device was properly configured[cite: 57].

However, it is important to note what happens after the `main()` function prints its message. In a bare-metal environment, there is no operating system to return to and provide a command prompt. A processor never stops; it constantly fetches and executes the next instruction in memory. To prevent the CPU from wandering into uninitialized memory and executing random (and potentially dangerous) garbage data, the program is forced into an infinite spin loop at the end of its execution[cite: 59]. This ensures the processor remains safely locked in a known state after the primary logic has concluded[cite: 59].

Chapter 2

Boot and CPU Configuration

2.1 Introduction to Low-Level CPU Control

The primary goal of this phase is to move beyond simply using the toolchain and to start writing real, low-level code to take direct control of the CPU. This involves understanding how the CPU starts up, how to configure it step by step, and what it truly means to run a system without an underlying operating system or standard C library[cite: 64].

The development process is structured around three core concepts:

- Understanding the memory layout and directing the linker script to place code and data in the correct physical locations[cite: 64].
- Controlling the CPU boot process through a startup script, which acts as a minimal bootloader.
- Operating in a freestanding environment to comprehend the limitations of bare-metal programming without standard libraries.

2.2 Memory Technologies in Embedded Systems

When designing an operating system or bare-metal software, it is crucial to understand the different memory technologies available on the hardware, as this dictates where instructions and variables are placed.

2.2.1 SRAM (Static RAM): The Working Memory

- SRAM sits directly on the CPU's memory bus, meaning every byte has a specific address and can be accessed using standard load/store instructions[cite: 66].
- It is extremely fast; on a microcontroller, a single access typically takes about one clock cycle[cite: 66].
- It is byte-writable, allowing the CPU to change single bytes at any time[cite: 66].
- It is volatile, meaning all data is lost when power is removed, and its contents are entirely undefined (random bits, not zeros) at power-on[cite: 66].

- Due to its writable nature, SRAM is the designated location for anything that changes during program execution, such as variables and the stack[cite: 66, 69].

2.2.2 NOR Flash: Execute-in-Place Memory

- Like SRAM, NOR Flash sits on the memory bus and can be addressed byte by byte, allowing fast random-access reads[cite: 67].
- Because it is on the address bus, the CPU can fetch and execute instructions directly from it without needing to copy the program to RAM first[cite: 67]. This feature is known as execute-in-place (XIP)[cite: 67].
- It is non-volatile, ensuring the program remains intact after power-off[cite: 67].
- Writing to NOR Flash is complex and cannot be done with simple store instructions[cite: 67]. It requires first erasing an entire sector (turning all bits to 1) and then programming specific words (turning bits from 1 to 0), a process that takes microseconds to milliseconds[cite: 67].
- Consequently, at runtime, NOR Flash is effectively treated as read-only memory[cite: 67]. It is used to store the code itself and read-only constants (like string literals)[cite: 69].

2.2.3 NAND Flash and Block Devices

- NAND Flash is the technology used in SSDs, hard disks, and SD cards[cite: 68].
- Unlike NOR Flash, it does not sit on the CPU's memory bus and is instead accessed through a separate controller[cite: 68].
- Data cannot be addressed byte by byte; it must be accessed in large blocks or pages (e.g., 512 bytes to 4 KB)[cite: 68].
- Because it lacks direct memory addressing, the CPU cannot fetch instructions from NAND Flash, nor can it read single variables directly[cite: 68].
- To execute a program stored on a NAND device, an OS loader must first copy the blocks into RAM, which is the standard process for desktop computers[cite: 68]. Microcontrollers generally skip this step by running their programs directly from NOR Flash[cite: 68].

2.2.4 Applying Constraints to C Code

Given the physical characteristics of these different memory technologies, an operating system must correctly place every element of a C program based on its requirements[cite: 69].

- **Code and constants:** Instructions and read-only data (like string literals) should remain in NOR Flash and be executed or read directly from there[cite: 69].

- **Variables (uninitialized):** Standard uninitialized variables that need to be updated at runtime must live in SRAM[cite: 69].
- **Variables (initialized):** Variables defined with an initial value present a unique challenge. Their initial starting value must be stored non-volatilely in NOR Flash so it survives a power cycle[cite: 69]. However, because they are variables and will change, that initial value must be copied from Flash into SRAM at boot time[cite: 69].

2.2.5 The Cortex-M3 Memory Map

To tell the compiler where to place these different elements, a developer must first thoroughly understand the hardware's memory map. On a 32-bit Cortex-M microprocessor, the CPU sees a single 4 GB physical address space[cite: 70]. The regions within this space are predefined by the architecture, though the specific sizes are determined by the board designer[cite: 70].

The standard layout is as follows[cite: 70]:

- **0x0000_0000 - 0x1FFF_FFFF:** This region is dedicated to code execution, typically where the NOR Flash is mapped[cite: 70]. On our emulated board, it accommodates 4 MB of storage starting at address 0x0[cite: 70].
- **0x2000_0000 - 0x3FFF_FFFF:** This region is designated for fast static RAM (SRAM)[cite: 70]. Our board maps 4 MB of RAM starting at 0x2000_0000[cite: 70].
- **0x4000_0000 - 0x5FFF_FFFF:** This area is reserved for memory-mapped I/O peripherals, such as UART or timers[cite: 70].
- **0x6000_0000 - 0xDFFF_FFFF:** This vast region is available for external RAM or other devices if the internal memory is insufficient[cite: 70].
- **0xE000_0000 - 0xE00F_FFFF:** These addresses map to the CPU's private internal registers (the System Control Block)[cite: 70]. They are accessed like memory locations but control the core behavior of the CPU itself[cite: 70].

Even though we are emulating this hardware in QEMU (where "Flash" is technically just a pre-loaded section of the host's RAM), the emulator enforces these access rules strictly: any attempt by the program to write to the simulated Flash region will result in an error[cite: 70].

2.2.6 The Linker Script: Mapping the Chip

The **Linker Script** (e.g., `scripts/mps2_m3.ld`) is the file where we describe the hardware's memory map to the toolchain and provide instructions on how to place our code within it[cite: 71].

The script is divided into core sections[cite: 71]:

- **MEMORY:** Defines which physical memory regions exist, their start addresses, and their sizes[cite: 71].

- **SECTIONS:** Instructs the linker on which input sections from the `.o` files should go into which output regions[cite: 71].
- **ENTRY:** Defines where execution starts[cite: 71].

Defining the Memory Regions

The **MEMORY** block dictates the layout of the available hardware. For example[cite: 72]:

```
MEMORY {  
    FLASH (rx) : ORIGIN = 0x00000000, LENGTH = 4M  
    RAM (rwx)  : ORIGIN = 0x20000000, LENGTH = 4M  
}
```

Each region declaration includes an identifier (e.g., **FLASH**), its starting address (**ORIGIN**), and its total size (**LENGTH**)[cite: 72]. The linker uses this information to verify that the compiled code fits; if the data assigned to a region exceeds its **LENGTH**, the link process will fail[cite: 72].

The letters in parentheses specify the access rights for that region[cite: 72]:

- **r:** Read
- **w:** Write
- **x:** Execute

For instance, **(rx)** indicates that the **FLASH** region can be read and executed, but not written to during normal runtime[cite: 72].

Defining the Memory Regions (Continued)

In the **MEMORY** block, we specify the initial parameters of each physical memory region based on the hardware constraints:

- We define the starting address of the **FLASH** region at `0x00000000`, which is the predefined start of the code section for Cortex-M3 processors[cite: 72].
- We define the starting address of the **RAM** region immediately after, at `0x20000000`, which is the designated space for static RAM on this architecture[cite: 72].

Every address is written in hexadecimal using 32 bits (8 characters). It is imperative that this description perfectly matches the physical hardware specifications. If a developer erroneously marks a region as writable (**w**) when the underlying technology is strictly read-only (like NOR Flash), the linker will silently generate the ELF file without raising errors. However, the program will inevitably crash at runtime when it attempts an illegal operation[cite: 72].

Symbol Assignments for the Stack Pointer

Within the linker script, we can define symbols that the linker will substitute with exact memory addresses during the linking process. A crucial example is calculating the starting address of the stack[cite: 72].

On ARM microprocessors, the stack grows downwards (backwards in memory)[cite: 72, 98]. Therefore, the most efficient approach is to set the initial stack pointer to the very end of the RAM[cite: 72]. In the script, this is done by defining a symbol (e.g., `_estack`) and assigning it the value of the RAM's starting address plus its total length[cite: 72]:

```
_estack = ORIGIN(RAM) + LENGTH(RAM);
```

Organizing Output Sections

The core function of the linker script is to declare which sections should exist in the final ELF image and what content should be placed inside them[cite: 73].

When compiling a C program composed of multiple source files, the compiler produces multiple object files (`.o`), each divided into various input sections. The linker script uses rules to aggregate these scattered input sections into unified output sections[cite: 73].

For example, to organize the executable code, we define a `.text` output section[cite: 73]:

```
.text : {  
    *(.text*)  
    *(.rodata*)  
    _etext = .  
} > FLASH
```

- `*(.text*)`: This command tells the linker to examine every input object file (`*`), find any section whose name starts with `.text`, and place its contents here[cite: 73]. The wildcard ensures that even if sections are slightly renamed by the compiler, they are correctly included[cite: 73].
- `*(.rodata*)`: This instructs the linker to take all read-only constants from all object files and append them immediately after the code within this same output section[cite: 73].
- `_etext = .`: The dot (`.`) represents the location counter (the current memory address)[cite: 73]. This line assigns the current address (which corresponds to the end of the `.text` and `.rodata` data) to a symbol named `_etext`[cite: 73]. This provides a programmatic way to know exactly where the code section finishes in memory[cite: 73].
- `> FLASH`: Finally, this specifies that the entire `.text` output section must be placed physically in the region labeled `FLASH` defined in the `MEMORY` block[cite: 73].

2.2.7 Run Address vs. Load Address: The .data Section

For read-only data and executable code (the `.text` section), memory placement is straightforward because they exclusively reside in Flash memory. However, writable global variables that are initialized (e.g., `int x = 5;`) require a different approach.

In the linker script, the `.data` section must be defined with two different memory addresses[cite: 74]:

- **VMA (Virtual Memory Address / Run Address):** This dictates where the variables will reside during execution, which must be in RAM so their values can be updated[cite: 74].
- **LMA (Load Memory Address):** This dictates where the initial values are permanently stored in the binary image, which must be in FLASH[cite: 74].

The hardware does not automatically move these bytes from the LMA to the VMA[cite: 74]. The linker script defines symbols for these boundaries (such as `_sidata` for the load address, and `_sdata` / `_edata` for the run addresses), but it is the responsibility of the boot code to manually copy the data from Flash to RAM before the program starts[cite: 74].

2.2.8 BSS, Heap, and Stack Layout

Other sections of the program must also be defined in the linker script to properly manage the RAM:

- **.bss Section:** This section contains uninitialized global variables. Unlike the `.data` section, it has no LMA content in Flash; the image only records its size[cite: 75]. It is assigned exclusively to RAM, and the boot code is responsible for filling this memory area with zeros using the `_sbss` and `_ebss` symbols[cite: 75].
- **Heap and Stack:** If the program utilizes dynamic memory allocation, a heap region must be defined. In standard embedded layouts, the heap grows upwards from the end of the `.bss` section, while the stack grows downwards from the very end of the RAM (`_estack`)[cite: 75].

Safety Asserts in the Linker Script

To ensure system stability, the linker script can include logical conditions using the `ASSERT` command[cite: 75]. For instance, an `ASSERT` can be programmed to verify that the predefined limits of the stack (`_StackLimit`) and the top of the heap (`_heap_top`) do not overlap[cite: 75]. If this mathematical condition is violated during compilation, the linker will immediately abort the process and throw an error, preventing a catastrophic memory corruption at runtime[cite: 75].

2.2.9 The Role of Linker-Defined Symbols

The primary reason we define numerous symbols in the linker script (like `_sdata` or `_ebss`) is that our startup code needs to know the precise addresses of the different memory regions to initialize them[cite: 76].

When the C or assembly code is initially compiled, the exact sizes of the sections (like how large the `.text` section will be) are completely unknown. Consequently, the starting address of the subsequent `.data` section is also unknown. By placing these symbols in the code, we instruct the toolchain to wait[cite: 76]. Once all object files are compiled and sized, the linker calculates the final memory layout and substitutes the symbolic placeholders with the correct, final physical addresses[cite: 76].

It is crucial to note that these linker-defined symbols do not hold storage or data themselves; the address *is* the information[cite: 76].

Memory Alignment

Another critical instruction often found in linker scripts is the `ALIGN` command. Many RISC processors, including the ARM Cortex-M family, require instructions and data to be aligned in memory (typically in 32-bit or 4-byte words). If a section ends at an address that is not perfectly divisible by four, the `ALIGN(4)` command automatically inserts padding (empty bytes) to skip to the next valid aligned address. Failing to align memory properly on these architectures will cause the CPU to misread instructions and immediately crash the software.

2.2.10 Reading the Compilation Results

Once an empty project or a basic C program is built, we can use specific tools provided by the toolchain to inspect the generated ELF file and understand exactly how the linker mapped the code and data into memory.

There are three primary ways to inspect the compilation output:

- **The Map File (`output.map`):** By passing the appropriate flags to the compiler, we instruct the linker to generate a map file[cite: 77]. This text file provides a comprehensive description of the entire memory configuration[cite: 77]. It lists all the memory regions, the output sections, and precisely which input sections (like the code coming from `main.o`) were placed inside them[cite: 77]. For example, inspecting the map file reveals how the read-only data (`.rodata`) is appended immediately after the executable code within the `.text` section[cite: 77].
- **The nm Tool:** Running the command `arm-none-eabi-nm -n` lists all the symbols in the compiled file, sorted by their memory addresses[cite: 77]. Next to each symbol is a letter indicating its kind or the section it belongs to[cite: 77]. Common letters include: **T** (or **t**) for text/code, **R** for read-only data, **D** for initialized data, **B** for BSS (uninitialized data), and **A** for absolute symbols that are not tied to a specific section[cite: 77].
- **The objdump Tool:** Running `arm-none-eabi-objdump -h` provides a summary of all sections, crucially displaying both the VMA (run address) and LMA (load address) for each[cite: 77]. When observing the `.data` section, the output explicitly confirms that the load address is in Flash while the run address is in RAM[cite: 78]. This proves that the ELF file contains the necessary information for the startup code to copy the initial variable values from Flash to RAM at boot[cite: 78].

2.2.11 Which Section Does a Declaration Go To?

To fully understand how the linker distributes data, we must add some global variables to our C program and observe where they land. The C language rules dictate how global declarations (outside any function body) are mapped to memory regions[cite: 79]:

- **Functions:** The body of a function goes into the `.text` section, residing permanently in Flash memory[cite: 79].
- **Constants:** Variables declared with the `const` keyword (e.g., `const int limit = 100;`) and string literals (e.g., `"hi"`) go into the `.rodata` section, which is also placed in Flash[cite: 79].
- **Initialized Variables:** Global variables with an explicit initial value (e.g., `int counter = 5;`) go into the `.data` section[cite: 79]. As analyzed previously, their initial values are stored in Flash, but they operate in RAM[cite: 79].
- **Uninitialized Variables:** Global variables without an initializer (e.g., `int total;`) or explicitly initialized to zero (e.g., `int zero = 0;`) go into the `.bss` section[cite: 79]. These live entirely in RAM and are zeroed out by the boot code at startup[cite: 79].

Pointers and Local Variables

It is important to distinguish between the content and the container when dealing with pointers. For example, the declaration `const char *greeting = "hi";` involves two distinct elements[cite: 79]:

1. The string literal `"hi"` itself is read-only, so it is placed in the `.rodata` section (Flash)[cite: 79].
2. The pointer variable `greeting`, which holds the memory address of that string, is a modifiable global variable. Therefore, it is placed in the `.data` section (RAM)[cite: 79].

Finally, local variables declared inside a function do not belong to any of these static linker sections[cite: 79]. They are dynamically allocated on the stack only while the function is executing, and their memory is released as soon as the function returns[cite: 79].

Try It: Predict, Then Check – Analyzing the Output

To observe how the linker handles different data types, we can declare several global variables in our code (such as `const int limit = 100;`) and run the `make` command. After building the project, we can inspect both the intermediate object file (`main.o`) and the finalized ELF file (`scratch-os.axf`) using the `nm` tool.

When analyzing the `limit` variable, a very interesting behavior emerges:

- In the object file (`main.o`), `nm` reports the `limit` variable with the letter **R**, indicating that it correctly sits in the read-only data section (`.rodata`)[cite: 81].

- However, in the final ELF image, `nm` reports the exact same variable with the letter `T`, indicating it belongs to the text section[cite: 81].

The reason for this transition lies within the linker script. The script explicitly instructs the linker to take all `.rodata` input sections from the various object files and place them immediately after the executable code inside the single `.text` output section[cite: 81]. Consequently, the bytes remain exactly the same and are still stored in Flash memory; only the name of the output section containing them has changed[cite: 81].

Locating String Literals in Memory

When defining a string via a pointer, such as `const char *greeting = "hi";`, two elements are created in memory: the pointer variable `greeting` and the string literal `"hi"` itself.

Because the string literal `"hi"` is a constant, it is placed in the `.rodata` section (and subsequently merged into `.text` in FLASH)[cite: 81]. However, unlike a defined variable, a string literal has no associated symbol name, which means the `nm` tool will never list it in its output[cite: 81].

Even though it lacks a symbol, the string physically exists in memory. To verify this, we can dump the raw binary contents of the `.text` section using `objdump`:

```
arm-none-eabi-objdump -s -j .text build/scratch-os.axf
```

[cite: 81]

This command dumps the memory of the specified section. The output displays the memory addresses and their corresponding hexadecimal byte values on the left, and the ASCII character equivalents on the right[cite: 81]. By carefully inspecting the ASCII column, one can visibly locate the characters forming the string `"hi"` (represented in hex as `68 69 00`, which corresponds to `'h'`, `'i'`, and the null terminator) sitting right after the `limit` variable in the Flash memory[cite: 81].

2.2.12 Advanced Memory Layout and Custom Sections

While the linker script generally handles the placement of standard C variables automatically, there are situations where a developer needs extreme flexibility. For example, if memory is extremely constrained, a developer might need to pack variables tightly into custom memory layouts to avoid losing bytes to alignment padding.

Forcing Variables into Specific Sections

You have the ability to force the compiler to place a variable into a specifically named section directly from your C code. This is done using compiler attributes. For instance:

```
int tagged __attribute__((section(".mySeg"))) = 42;
```

This command defines an integer variable named `tagged` and forces the linker to store it in a custom section named `.mySeg`. In the linker script, you can then define exactly where this `.mySeg` section should be placed physically, giving you absolute control over the memory layout. Even if you do not explicitly define `.mySeg` in the linker script, the linker will still attempt to manage it as an "orphan" section, though it's best practice to define it explicitly.

2.2.13 Managing Memory Size Constraints

The linker script is also the definitive source of truth for the physical memory limits of the board. In our script, we initially defined the `FLASH` region with a length of 4 Megabytes.

If we want to emulate a board with significantly less physical memory (for example, a very small flash memory of only 100 bytes), we must change the script:

```
FLASH (rx) : ORIGIN = 0x00000000, LENGTH = 100
```

To test this change, we must force the `Makefile` to rebuild everything. Because `Make` typically only tracks changes in the `.c` or `.h` source files and not the linker script, we use the `-B` flag (e.g., `make -B`) to force a complete recompilation.

If the compiled code requires more than 100 bytes, the linker will immediately throw a size error, stating that the `FLASH` region overflowed. This demonstrates how the linker uses the script to actively protect against generating firmware that would physically not fit on the target device.

Enforcing Memory Constraints and Overflows

The linker script strictly enforces the physical memory limits defined in the `MEMORY` block[cite: 87]. If the generated code size exceeds the defined Flash capacity—for instance, if the Flash is artificially restricted to 100 bytes but the code requires 128 bytes—the linker performs a static check and halts the compilation[cite: 87]. It will throw an explicit overflow error (e.g., "region 'FLASH' overflowed by 28 bytes")[cite: 87]. This compile-time safeguard is critical; without it, executable code could blindly spill over into the RAM section, silently overwriting variables and causing catastrophic failures[cite: 87]. Resolving a legitimate Flash overflow requires either optimizing the code to reduce its footprint or upgrading the hardware to a board with more memory.

Conversely, if we reduce the RAM size (for example, down to 8 KB), a different type of error may occur[cite: 87]. Unlike Flash memory, which is filled statically, RAM accommodates dynamic regions like the heap and the stack. The error generated in this scenario is triggered by the explicit `ASSERT` command written in the linker script, which actively checks if the stack limit and the top of the heap overlap[cite: 87]. If a developer removes this `ASSERT` safeguard, the linker will successfully produce an executable file without warnings, but the program will inevitably crash at runtime when the stack overwrites the heap data[cite: 87]. Writing a robust linker script with mathematical assertions is essential for preventing these silent runtime failures.

2.2.14 Reset: The Hardware Contract

Once the memory layout is finalized and the ELF file is loaded into memory, the machine can be powered on[cite: 88]. Operating systems on standard desktop computers rely on a BIOS, a loader, and a C runtime to initialize the environment. However, on a bare-metal ARM Cortex-M microprocessor, absolutely none of these services exist[cite: 88].

When the Cortex-M processor comes out of reset, the hardware executes a strictly predefined and immutable sequence[cite: 88]. Before any user code runs, the hard-

ware performs exactly two memory reads at the very beginning of the address space (0x00000000)[cite: 88]:

1. **Word 0:** It reads the first 4 bytes at address 0x0 and copies this value directly into the Stack Pointer (SP) register[cite: 88].
2. **Word 1:** It reads the next 4 bytes at address 0x4 and copies this value into the Program Counter (PC) register, jumping to that address to begin execution[cite: 88].

This behavior is a hardwired contract[cite: 88]. The CPU blindly trusts that the developer has placed the correct initial stack address and the correct starting address of the application code into these first eight bytes of memory[cite: 88]. Everything else—copying the `.data` section to RAM, zeroing out the `.bss` section, and setting up the C environment—must be explicitly handled by the developer’s boot code starting at the address provided in Word 1[cite: 88].

2.2.15 The Vector Table: Controlling the Boot Process

To ensure the hardware jumps directly to our main program after loading the stack pointer, we must properly construct the **Interrupt Vector Table**[cite: 89].

The vector table is a predefined array of 32-bit addresses (slots) placed at the very beginning of the memory space (starting at address 0x00000000)[cite: 89]. Each slot has a specific architectural meaning, instructing the CPU where to jump when a specific exception or interrupt occurs[cite: 89].

For the initial boot process, we are primarily concerned with the first two slots:

- **Slot 0 (Initial SP):** This slot must contain the starting address of the stack[cite: 89]. We fulfill this by placing the linker-defined symbol `_estack` here[cite: 89].
- **Slot 1 (Reset_Handler):** This slot must contain the memory address of the first instruction to execute upon reset[cite: 89]. We fulfill this by placing a symbol (typically named `Reset_Handler`) that points to our boot code or the beginning of our `main` function[cite: 89].

Implementing the Vector Table in Code

We must explicitly create this table in our source code (often in an assembly file like `boot/startup.s`) and ensure the linker places it correctly.

We define a specific section in the code, for example, `.isr_vector`, and place the two required words inside it[cite: 90]. When the object file is created, it allocates space for these two words and stores the symbols (`_estack` and `Reset_Handler`)[cite: 90]. During the linking phase, the linker script—which dictates that the `.isr_vector` section must be placed at the very beginning of the Flash memory—replaces these symbols with their final, absolute physical addresses[cite: 90].

When the compiled ELF file is subsequently flashed onto the board, the table sits precisely at address 0x0, ready for the hardware to read it at power-on, ensuring a flawless boot sequence[cite: 90].

Protecting Low-Level Code from Optimization

Writing low-level embedded software requires careful management of compiler and linker optimizations. These tools are highly aggressive at stripping away "dead" code[cite: 91].

Because the vector table is read directly by the hardware and never explicitly called by any C or assembly instruction within the program, the compiler might erroneously flag it as unused and delete it during the optimization phase[cite: 91]. If the vector table is removed, the board will fail to boot completely[cite: 91].

To prevent this, developers must use specific directives in the linker script (such as the `KEEP()` command) or specific compiler flags to explicitly instruct the toolchain that these memory locations are critical and must be retained in the final binary, regardless of whether they appear to be unused by the software[cite: 91].

2.3 Taking Control of the CPU

2.3.1 Startup Code: Assembly vs. C

The previous sections established how to correctly place code and data into memory. The next critical phase is understanding precisely what code must be placed there to initialize the system and take full control of the CPU. The low-level startup code is essential for bridging the gap between hardware power-on and the execution of the main C program.

While this startup sequence can technically be written in C, it is highly instructive—and often more straightforward—to write it directly in Assembly language[cite: 93]. Writing this sequence in C requires complex abstractions and pointer arithmetic to manipulate low-level hardware states[cite: 93]. Assembly, conversely, provides a transparent 1:1 mapping between the instruction written and the physical action taken by the CPU, making the core initialization steps much clearer to understand[cite: 93].

2.3.2 Remote Debugging in a Bare-Metal Environment

Developing low-level software introduces severe debugging challenges. In standard high-level programming, developers frequently rely on `printf` statements to output variables and trace execution flow. In a bare-metal environment during the boot process, there is no operating system, no initialized C library, and no configured output device, rendering `printf` completely unusable[cite: 94].

To investigate crashes or verify hardware states at this level, a **Debugger** is strictly required[cite: 94]. Specifically, a remote debugging architecture is necessary, where the debugger operates across two separate terminals:

1. **The Target:** This is the embedded platform (in our case, the QEMU emulator) where the compiled code actually executes[cite: 94]. It runs a minimal GDB server that listens for incoming connections[cite: 95].
2. **The Host:** This is the developer's PC, running the debugging interface[cite: 94]. It connects to the target (e.g., via TCP port 1234 when using QEMU, or via a special USB debug cable on real hardware) and issues commands[cite: 95].

When the developer issues a command on the host (like "step instruction"), the command is transmitted to the target board, which executes the specific action and then pauses, returning control and the updated hardware state back to the host[cite: 94, 95].

GDB: The GNU Debugger

The standard tool for this task is **GDB** (the GNU Debugger)[cite: 94]. While it lacks a graphical user interface and operates entirely via the command line, it is exceptionally powerful and specifically designed for this client-server architecture[cite: 94]. To use it with our ARM target, we run the cross-compiled version (`arm-none-eabi-gdb`) and load our ELF file (`scratch-os.axf`) so the debugger has access to all the symbol names (like `main` or `Reset_Handler`)[cite: 95].

We then connect to the running emulator using the command `target remote :1234`. From there, GDB allows us to inspect CPU registers (`info registers`), examine raw memory addresses in hexadecimal (`x/wx`), step through the program one single machine instruction at a time (`stepi`), and set breakpoints to pause execution at specific labels (`break`).

Setting Up the Debugging Session

To use the debugger in practice, we must coordinate both ends of the connection:

1. Preparing the Target (QEMU): Instead of running the emulator normally, we must instruct it to start in debug mode. In the `Makefile`, there is a specific target named `debug` which launches QEMU with additional flags:

```
-s -gdb tcp::1234
```

The `-s` flag is critical: it commands the emulator to freeze the CPU immediately at reset, before the very first instruction is executed[cite: 95]. The `-gdb tcp::1234` flag opens a GDB server listening on port 1234[cite: 95]. When we type `make debug`, the terminal will show no program output; the emulator appears stalled because the CPU is powered on but intentionally waiting for a debugger to connect.

2. Connecting the Host (GDB): In a second terminal, we launch the debugger. It is vital to pass the ELF file as an argument when starting GDB (e.g., `arm-none-eabi-gdb build/scratch-os.axf`)[cite: 95]. GDB does not execute this file; it reads the metadata inside it[cite: 95]. Because we compiled our code with flags that preserve debugging symbols, GDB can map raw hexadecimal memory addresses back to human-readable C variables and function names (like `main`)[cite: 95]. Without the ELF file, the developer would only see raw addresses and would have to manually track the memory locations of every variable.

Note on Security: Keeping debugging symbols is essential during development. However, before releasing a final commercial product, these symbols are always stripped from the ELF file. Leaving them in provides malicious actors with a detailed blueprint of how the code operates, making it trivial to exploit.

Executing Remote Commands

Once GDB is running, we connect to the frozen target by typing `target remote :1234`[cite: 95]. At this exact moment, the CPU has just completed its hardware

reset contract: it has read the initial Stack Pointer (SP) and the Program Counter (PC) from the vector table, and is paused exactly at the `Reset_Handler`.

When a developer types a command into GDB (for instance, `info registers sp pc` to inspect the stack and program counters), a complex transaction occurs:

1. GDB translates the human-readable command into the appropriate debugging protocol.
2. The request is sent over the communication link (in this case, a simulated TCP port, but on real hardware, this would be a physical USB, Ethernet, or JTAG debug cable).
3. A dedicated hardware debug circuit on the target board receives the request and translates it into internal CPU signals.
4. The CPU's internal debug unit reads the requested registers, packs the data back into the protocol, and sends it across the link.
5. GDB receives the raw data, formats it into human-readable text, and prints it on the developer's screen.

The Initial PC Value and the Vector Table

When we inspect the program state immediately after reset, the Program Counter (PC) holds a specific value (e.g., `0x8` or `0xA`). This is not the address of the first instruction, but the address *after* the vector table.

As established, the first part of the memory contains the vector table. In a minimal setup with only the initial SP and the Reset Handler, there are two entries. Since each entry is a 32-bit word (4 bytes), the table occupies the first 8 bytes (addresses `0x0` through `0x7`). Therefore, the very first line of actual executable code sits exactly at address `0x8`. GDB intelligently reads the ELF file and associates this raw address with the `Reset_Handler` symbol, confirming the hardware executed the boot sequence perfectly.

2.3.3 Cortex-M3 Programmer's Model: The Registers

To write the assembly startup script, one must understand the CPU's internal architecture, specifically the registers available to the programmer. All registers in the Cortex-M3 are 32 bits wide. At the assembly level, there are no "variables"; data manipulation occurs strictly within these registers and memory.

General Purpose Registers

- **R0 to R12:** These are general-purpose registers used for storing variables and performing calculations. Registers `R0` through `R3` hold a special significance in the C calling convention, as they are used to pass the first four arguments to a function.

Special Registers

- **R13 (SP - Stack Pointer):** Holds the memory address of the top of the stack. As noted earlier, the stack on ARM processors grows downwards (backwards in memory). When data is pushed, the CPU first decrements the SP by 4 bytes, then writes the data.
- **R14 (LR - Link Register):** A critical register for program flow. When a function is called, the hardware saves the return address (the address of the instruction following the call) into the LR. The assembly code is responsible for managing this register properly, especially during nested function calls, to ensure the program can return correctly.
- **R15 (PC - Program Counter):** Holds the address of the next instruction the CPU will execute.

The Status Register (xPSR)

The Program Status Register (xPSR) packs multiple pieces of crucial state information into a single 32-bit register:

- **Flags (Bits 31-28):** Contains the condition flags (Negative, Zero, Carry, overflow - NZCV). These are set by arithmetic operations or specific compare (`cmp`) instructions and are used to evaluate conditional branches (e.g., `if` statements).
- **Thumb Bit (Bit 24):** ARM processors have two instruction sets: standard ARM (32-bit) and Thumb (16-bit for compactness). The Cortex-M series exclusively supports the Thumb instruction set. Therefore, this bit must always be 1; if it is ever cleared to 0, the CPU will throw a fault.
- **Exception Number (Bits 8-0):** When a hardware problem or interrupt occurs (an exception), the CPU stops normal execution. The reason for the stop is encoded as an 8-bit number and written here. If these bits are 0, the CPU is running normal code (Thread mode); if they hold a non-zero value, the CPU is currently executing an exception handler (Handler mode).

2.3.4 ARM Assembly Basics: A Load/Store Architecture

The ARM architecture is strictly a load/store machine. This means the CPU cannot perform arithmetic or logical operations directly on data stored in RAM.

To manipulate data, the programmer must follow a strict three-step process:

1. **Load (`ldr`):** Move the data from the memory address into a CPU register.
2. **Operate:** Perform the math or logic operation entirely within the CPU registers.
3. **Store (`str`):** Move the result from the CPU register back into the memory address.

This fundamental limitation dictates the shape of all ARM assembly code.

Load/Store and Move Instructions in Detail

Because the ARM architecture cannot perform mathematical operations directly on memory, data movement is the foundation of every assembly script. The two primary instructions are:

- **ldr (Load Register):** Moves data from memory into a CPU register.
- **str (Store Register):** Moves data from a CPU register into a memory location.

These instructions come in various flavors depending on the addressing mode used (e.g., direct address, post-incrementing pointers).

In addition to memory operations, the CPU frequently needs to move data between its own internal registers. This is handled by the `mov` instruction (or `movs` if the operation should also update the condition flags).

Example Snippet:

```
ldr r0, =_estack
mov sp, r0
movs r3, #0
```

In this snippet, `=_estack` indicates an address defined in the linker script. The first instruction loads that 32-bit address into R0. The second instruction copies that address from R0 into the Stack Pointer (SP). The third instruction loads the immediate constant value 0 into R3.

The 32-bit Instruction Encoding Constraint

A fundamental constraint of the ARM architecture is that every instruction must be encoded into a fixed length of 32 bits (or 16 bits in Thumb mode). This creates a problem when trying to load a 32-bit constant or address into a register.

It is mathematically impossible to fit a 32-bit constant, the instruction opcode, and the destination register identifier all into a single 32-bit instruction space. Therefore, immediate values (constants embedded directly in the instruction, denoted by `#`) are strictly limited to small sizes (usually 8 or 12 bits).

To load full 32-bit addresses (like `=_estack`), the assembler uses a trick called the **literal pool**. The assembler stores the full 32-bit constant in a nearby table in the code section and then issues an instruction to read that constant from the table relative to the Program Counter (PC). In GDB, an instruction written as `ldr r0, =value` will actually appear as something like `ldr r0, [pc, #92]`.

2.3.5 Control Flow: Decisions and Loops

Branching and loops in assembly rely entirely on the `xPSR` status register. A decision is always a two-step process: first, an instruction compares values and sets the condition flags; second, a conditional branch instruction jumps based on those flags.

Example: A while Loop in Assembly Consider a C loop: `while (r1 < r2) { ... }`

In assembly, this is implemented as a test at the top and an unconditional jump at the bottom:

```

loop:
    cmp r1, r2
    bhs done
    @ ... loop body ...
    b loop
done:

```

Here, `cmp r1, r2` computes `r1 - r2` and updates the flags. `bhs done` stands for "Branch if Higher or Same"; if `r1 ≥ r2` (unsigned), the loop terminates and jumps to `done`. Otherwise, execution falls through to the loop body, and the unconditional branch `b loop` sends it back to the test.

2.3.6 Assembler Directives

Unlike C code, which uses preprocessor directives like `#include`, assembly files (`.s`) use assembler directives. These are instructions for the assembler itself rather than instructions for the CPU to execute.

In GNU Assembly, directives always begin with a dot (`.`). They are used to control memory allocation, section definitions, and symbol visibility.

Example Directive Block:

```

.section .isr_vector, "a", %progbits
.global isr_vector
isr_vector:
    .word _estack
    .word Reset_Handler

```

- **.section:** Tells the assembler to place what follows into the specified section (e.g., `.isr_vector` or `.text`).
- **.global:** Makes a label visible to the linker. Without this, a label acts like a static C variable, visible only within that specific file.
- **.word:** Allocates exactly one 32-bit word of memory and stores the given value (or address) inside it.
- **.thumb_func:** (Used in text sections) Tells the assembler that the following label points to Thumb code. The assembler will automatically set the lowest bit (bit 0) of the label's address to 1, which the CPU requires to remain in Thumb mode.

While it is technically possible to replicate these directives in C using complex compiler attributes (e.g., `__attribute__((section(".isr_vector")))`), writing it directly in assembly is generally clearer and more precise for initial boot sequences.

2.4 Writing the Startup Script (Reset_Handler)

The `Reset_Handler` is the very first piece of code executed by the CPU. Its job is to turn a bare, uninitialized hardware chip into an environment fully capable of running a standard C program.

The startup plan follows a strict 10-step sequence:

1. **Mask Interrupts:** Disable all interrupts. If a hardware device triggers an interrupt before the system is configured, the CPU will crash.
2. **Set the Stack Pointer (SP):** Ensure the C environment has a working stack.
3. **Copy .data from Flash to RAM:** Initialize global variables that have explicit starting values.
4. **Zero the .bss section:** Keep the C promise that uninitialized globals start at zero.
5. **Configure the CPU (Steps 5-8):** Set up vector table addresses, fault handlers, and priorities.
6. **Unmask Interrupts:** Re-enable hardware interrupts now that the system is safe and configured.
7. **Jump to main():** Hand control over to the C application.

2.4.1 Steps 1 and 2: Mask Interrupts and Set SP

When the CPU boots, there is no guarantee what the hardware peripherals are doing. To prevent a catastrophic interruption during the fragile setup phase, the first instruction must mask (disable) interrupts using the `cpsid i` command.

```
cpsid i           @ Step 1: Mask interrupts
ldr r0, =_estack  @ Step 2: Set SP
mov sp, r0
```

Note on the Stack Pointer: Even though the hardware automatically loads the SP from Word 0 of the vector table at reset, it is a highly recommended practice to set it explicitly in the `Reset_Handler`. A bootloader, or a debugger like GDB, might jump directly to the `Reset_Handler` bypassing the hardware reset sequence. Explicitly setting SP guarantees the code works flawlessly in all scenarios. Note that SP is a special register, so the address must first be loaded into a general-purpose register (like R0) before being moved to SP.

2.4.2 Step 3: Copy .data from Flash to RAM

After setting the Stack Pointer, the startup script must initialize the global variables that have explicit starting values (the `.data` section). As defined in the linker script, these variables reside in RAM during runtime (VMA), but their initial values are permanently stored in Flash memory (LMA).

The goal is to copy the data byte-by-byte (or word-by-word) from the Flash load address to the RAM run address.

To achieve this, the code must use the linker-defined symbols:

- `_sidata`: The start of the data in Flash (Source).
- `_sdata`: The start of the data in RAM (Destination).
- `_edata`: The end of the data in RAM (Limit).

The Copy Loop in Assembly

The logic is equivalent to a simple array copy in C: `while (dst < end) { *dst++ = *src++; }`

Here is how it translates to ARM assembly:

```

    ldr r0, =_sidata      @ src: initial values in FLASH
    ldr r1, =_sdata       @ dst: start of data in RAM
    ldr r2, =_edata       @ end: end of data in RAM

copy_data:
    cmp r1, r2            @ while (dst < end)
    bhs copy_data_done    @ If dst >= end, exit loop

    ldr r3, [r0], #4      @ Load word from src, then src += 4
    str r3, [r1], #4      @ Store word to dst, then dst += 4
    b copy_data           @ Repeat loop

copy_data_done:
```

Post-Increment Addressing

A key feature of the ARM instruction set demonstrated here is the post-increment addressing mode: `[r0], #4`. This syntax tells the CPU to perform two operations in a single instruction:

1. Load the 32-bit word from the memory address currently in R0 into the destination register (R3).
2. Immediately increment the address in R0 by 4 bytes (since one 32-bit word = 4 bytes).

This allows the pointer to automatically step to the next word in memory without requiring a separate `add r0, r0, #4` instruction, making the code both smaller and faster.

This exact block of code must be pasted into the `boot/startup.s` file under the designated "Steps 1-9" section to ensure the `.data` section is properly initialized before `main()` executes.

Try It: Bring Data to Life

Once Step 3 (copying the `.data` section) is added to `boot/startup.s` and the code is recompiled, running the emulator will yield a noticeable difference.

Before this step, any initialized global variables (like an array containing a string) resided in RAM as zeros, meaning they appeared empty when the program attempted to print them. After implementing the copy loop, the program will successfully print both strings, proving that the initial bytes were successfully migrated from Flash to RAM before `main` was executed.

Debugging the Boot Process

It is highly recommended to execute this startup code step-by-step using GDB. By setting a breakpoint and using the `stepi` (step instruction) command, a developer can watch the CPU execute the loop in real-time. By inspecting the registers (e.g., `p/x $r1`), you can see the destination pointer incrementing by 4 bytes on each iteration (e.g., climbing from `0x20000000`, to `0x20000004`, etc.) until it equals the end address, providing absolute confirmation of how the hardware works.

Try It: Break the Stack on Purpose

To fully understand the fragility of the bare-metal environment, developers should purposefully break the configuration and observe the resulting errors.

Test 1: Stack Outside of RAM If the developer changes the instruction in Step 2 to load an address far outside the physical RAM boundaries (e.g., `ldr r0, =0x30000000`), the program will crash immediately. When the C program starts and attempts to push variables or return addresses onto the stack, it writes to a non-existent physical address. The CPU attempts to report this memory fault, but the fault reporting mechanism itself requires a working stack to push the exception context. This impossible situation causes the CPU to give up and enter a terminal "Lockup" state.

Test 2: Unaligned Stack Pointer The Stack Pointer must always be aligned to a 4-byte boundary. If a developer manually forces the SP to an odd address (e.g., `0x20000001`), the CPU will silently drop the two lowest bits to force alignment. This means the SP will effectively jump to an unexpected location, causing writes to be ignored or data to be corrupted, leading to silent failures where functions crash without any explicit error messages. Recognizing these specific failure modes is a critical skill for embedded debugging.

2.4.3 Step 4: Zero the .bss Section

The fourth step in the boot sequence is to initialize all uninitialized global variables to zero[cite: 48]. These variables reside in the `.bss` section between the `_sbss` and `_ebss` bounds[cite: 48]. While uninitialized memory natively holds random values at power-on, the C standard guarantees that uninitialized globals start at zero[cite: 48]. Furthermore, actively zeroing out this memory is a good security practice to wipe any residual data left by previous programs.

To accomplish this, the startup script uses the `_sbss` and `_ebss` symbols defined in the linker script to determine the start and end of the region[cite: 48]. Because the ARM architecture does not allow writing immediate values directly to memory, the value 0 must first be stored in a register (e.g., `R3`)[cite: 48].

```

    ldr r1, =_sbss      @ dst: start of .bss
    ldr r2, =_ebss      @ end: end of .bss
    movs r3, #0         @ value to write

zero_bss:
    cmp r1, r2          @ while (dst < end)
    bhs zero_bss_done

```

```
    str r3, [r1], #4      @ *dst++ = 0;
    b zero_bss
```

zero_bss_done:

[cite: 48]

2.4.4 Exceptions and Faults: Completing the Vector Table

Up to this point, the code has used a minimal vector table containing only two entries: the initial Stack Pointer (Slot 0) and the `Reset_Handler` (Slot 1)[cite: 51]. However, the ARM CPU expects a complete vector table consisting of at least 16 slots (occupying 64 bytes in total)[cite: 51].

An exception is any hardware event that forces the CPU to stop executing the current code and jump to a specific handler function in Handler mode[cite: 50]. Apart from the first two slots, all remaining slots in the vector table are associated with these exceptions[cite: 50]:

- **Slot 2 (NMI - Non-Maskable Interrupt):** This slot contains the address of the handler for non-maskable hardware interrupts[cite: 50]. If the CPU receives an NMI signal, it immediately jumps to the address stored here.
- **Slot 3 (HardFault):** This acts as a fallback or "wildcard" fault handler[cite: 50]. If any hardware error occurs (such as memory corruption or an invalid instruction) and there is no specific handler defined for it, the CPU escalates the problem to a HardFault[cite: 50]. In robust operating systems, the `HardFault_Handler` typically prints a "kernel panic" message or dumps the memory state to assist developers in debugging the fatal error.

Specific Fault Handlers

Beyond the general HardFault, the Cortex-M architecture supports specific fault handlers for different types of hardware or execution errors[cite: 50, 58]. Proper configuration of these handlers is crucial for both debugging and security:

- **Slot 4 (MemManage):** If the CPU is equipped with a Memory Protection Unit (MPU) and the code attempts to access a protected or invalid memory region, it triggers a Memory Management fault[cite: 50, 58].
- **Slot 5 (BusFault):** This handler is triggered if there is an error on the system bus (e.g., trying to read an address where no hardware responds)[cite: 50, 58].
- **Slot 6 (UsageFault):** This exception is raised due to invalid code execution, such as attempting to execute a malformed instruction or dividing by zero[cite: 50, 58]. From a security perspective, properly handling UsageFaults is critical. If a hacker injects illegal instructions and the software lacks a proper handler, the CPU might jump to a random memory location where malicious payload code has been placed.

Implementing the Vector Table with Weak Symbols

To create a fully configured system, the assembly script must define a word in the `.isr_vector` section for every possible exception slot (up to slot 15 for the core Cortex-M exceptions)[cite: 51]. Each word must contain the address of a handler function.

While it is possible to point all exceptions to a single, endless loop to simply halt the CPU upon failure, a much better practice is to use a generic template leveraging **weak symbols** and aliases[cite: 51].

```
.word HardFault_Handler

.weak HardFault_Handler
.thumb_set HardFault_Handler, Default_Handler
```

```
Default_Handler:
    b Default_Handler      @ for (;;) {}
```

[cite: 51]

How it works:

1. `.word HardFault_Handler` places the address of the `HardFault_Handler` label into the table slot[cite: 51].
2. `.thumb_set HardFault_Handler, Default_Handler` tells the assembler: "If you cannot find a specific function named `HardFault_Handler`, use the address of `Default_Handler` instead." [cite: 51]
3. `.weak HardFault_Handler` is the crucial element[cite: 51]. It tells the linker that this assignment is a fallback default[cite: 51]. If a developer writes a normal, non-weak C function named `void HardFault_Handler(void)` anywhere in their high-level code, the linker will ignore the assembly default and route the exception directly to the developer's C function[cite: 51].

This architecture allows systems programmers to write a complex, robust assembly startup script once, and never touch it again. Application developers can selectively override specific handlers (like `HardFault_Handler`) in their C code without needing to modify the low-level assembly[cite: 51].